SparseSignals

Jimmy Wu

2026-03-29

Introduction

High-dimensional data have become increasingly common in modern statistics and data science. In many modern applications, the number of potential predictors is extremely large relative to the number of observations, creating situations in which classical statistical methods become inefficient, unstable, or even impossible to apply. Text data, genomics, online behavior, and other contemporary datasets often produce thousands or tens of thousands of features after preprocessing. In these settings, classical methods often become unstable, computationally expensive, and difficult to interpret. As a result, modern statistical learning relies on methods designed for high-dimensional data, including sparse representations, regularization, dimension reduction, and nonlinear learning algorithms (Hastie, Tibshirani, and Friedman).

This project uses the *SMS Spam Collection Data Set*, which contains 5,572 text messages labeled as spam or ham. After converting the messages into a TF–IDF document-term matrix, the dataset becomes high-dimensional and sparse because each message contains only a small subset of all possible words. Sparse matrix representations are therefore useful because they reduce storage requirements and make computation more efficient (Manning, Raghavan, and Schütze).

To study scalable learning methods, this project first applies penalized classification models, including logistic, ridge, and lasso regression. These methods are appropriate for high-dimensional settings as ridge regression stabilizes coefficient estimates, while lasso additionally performs variable selection (James et al.). The project then applies dimension reduction techniques, including truncated SVD and random projections, to investigate whether predictive performance can be maintained with fewer features (Bingham and Mannila).

Finally, the project explores kernel-based methods to capture nonlinear relationships that may not be handled well by linear models. Comparing these methods provides insight into the trade-offs among predictive accuracy, interpretability, and computational efficiency (Cortes and Vapnik).

Methods

The SMS messages were first converted into a text corpus and preprocessed by converting all words to lowercase, removing punctuation, removing common stop words, and constructing a TF-IDF document-term matrix. Because most messages contain only a small fraction of all possible words, the resulting matrix is highly sparse. The dataset was then divided into training and testing sets using an 80/20 split. All models were fit using the training set and evaluated on the testing set. Model performance was measured primarily by classification accuracy, precision, recall, F1-score, and computational time.

To examine the effect of regularization, three linear classification methods were compared: Logistic regression, Ridge regression, and Lasso regression. Ridge and lasso regression used cross-validation to choose the optimal penalty parameter λ. The ridge model was included because it stabilizes coefficient estimates in high-dimensional settings, while lasso was included because it can additionally perform variable selection.

Next, two dimension reduction approaches, Truncated singular value decomposition (SVD) and Random projection were applied before fitting a logistic regression model. For truncated SVD, the TF–IDF matrix was reduced to a smaller number of components while retaining most of the variation in the data. Random projection was then used as an alternative low-dimensional representation to compare whether a simpler and more computationally efficient reduction method could achieve similar results. Finally, a support vector machine with a radial basis function kernel was fit to the reduced data. This model was included to determine whether allowing nonlinear decision boundaries would improve predictive performance compared with the linear methods.

Results

Data Exploration

Before comparing the models, we first explore the structure of the dataset and the resulting TF-IDF matrix.

```
sms <- read.csv("spam.csv", stringsAsFactors = FALSE)

sms <- sms %>%
  select(label = v1, text = v2)

dim(sms)
```

```
## [1] 5572    2
```

```
table(sms$label)
```

```
##
##  ham spam
## 4825  747
```

```
prop.table(table(sms$label))
```

```
##
##       ham      spam
## 0.8659368 0.1340632
```

The dataset contains 5572 text messages and 2 variables, which are the message label and the message text itself. Among these observations, 4825 messages are labeled as *ham* (non-spam) and 747 are labeled as *spam*. Thus, approximately 86.6% of the messages are *non-spam*, while only 13.4% are *spam*.

This class imbalance is important because it means that a model could obtain relatively high accuracy simply by predicting most messages as ham. Therefore, in later sections, accuracy alone will not be sufficient to evaluate model performance. Additional measures such as precision, recall, and F1-score will be necessary to determine how well each method identifies spam messages specifically.

```
sms$text <- iconv(sms$text, from = "latin1", to = "UTF-8", sub = "")

corpus <- VCorpus(VectorSource(sms$text))

# Preprocessing
```

```r
corpus <- tm_map(corpus, content_transformer(tolower))
corpus <- tm_map(corpus, removePunctuation)
corpus <- tm_map(corpus, removeNumbers)
corpus <- tm_map(corpus, removeWords, stopwords("english"))
corpus <- tm_map(corpus, stripWhitespace)

# Construct TF-IDF document-term matrix and convert to ordinary matrix
dtm <- DocumentTermMatrix(
  corpus,
  control = list(weighting = weightTfIdf)
)

dtm_matrix <- as.matrix(dtm)

dim(dtm_matrix)
```

```
## [1] 5572 8235
```

```r
# Sparsity
nonzero <- sum(dtm_matrix != 0)
total <- length(dtm_matrix)
sparsity <- 1 - nonzero / total

nonzero
```

```
## [1] 43994
```

```r
total
```

```
## [1] 45885420
```

```r
sparsity
```

```
## [1] 0.9990412
```

The TF-IDF document-term matrix contains 5572 messages and 8235 unique terms. Although the original dataset contains only 5,572 observations, preprocessing the text expands the predictor space to more than eight thousand features due to the high-dimensional nature of the problem.

At the same time, the matrix is extremely sparse. Out of 45,885,420 total entries, only 43,994 are nonzero. This yields a sparsity of approximately 99.9%, meaning that nearly all entries in the matrix are zero.

The extremely high sparsity justifies the use of sparse matrix methods and motivates the comparison of scalable learning techniques in later sections. In particular, regularization and dimension reduction are likely to be important because the number of predictors is much larger than the amount of information contained in each message.

```r
# Most common terms in the TF-IDF matrix
term_freq <- colSums(dtm_matrix)

top_terms <- sort(term_freq, decreasing = TRUE)[1:15]

top_terms
```

```
##     call      now     will      can      ill      get     come     just
## 244.8353 208.4832 205.0407 195.5952 184.0987 183.9363 179.7268 168.9689
##     ltgt      lor     home     good     dont     like      got
## 165.2087 160.3955 153.1546 150.1502 145.0116 144.8433 144.2131
```

The most common terms in the TF-IDF matrix include words such as call, now, will, can, ill, indicating that a relatively small set of words appears frequently across the messages. Several of these words are likely associated with common *spam* themes such as promotions, prizes, free offers, or urgent requests.

Although terms help distinguish *spam* from *non-spam* messages, many words still appear only rarely, which further contributes to the extreme sparsity of the feature matrix. This pattern suggests that methods capable of handling many weak or infrequent predictors, such as ridge and lasso regression, may perform better than ordinary logistic regression.

Next, we split the data into training and testing sets and fit the first three models: logistic regression, ridge regression, and lasso regression.

```r
set.seed(20260416)
sms$label <- factor(sms$label)

train_index <- createDataPartition(sms$label, p = 0.8, list = FALSE)

x_train <- dtm_matrix[train_index, ]
x_test  <- dtm_matrix[-train_index, ]

y_train <- sms$label[train_index]
y_test  <- sms$label[-train_index]

# Keep only the 100 most common terms to eliminate run time
term_totals <- colSums(x_train)
top100 <- order(term_totals, decreasing = TRUE)[1:100]

x_train_small <- x_train[, top100]
x_test_small  <- x_test[, top100]

# Logistic regression
log_model <- glm(
  y_train ~ .,
  data = data.frame(y_train, x_train_small),
  family = "binomial"
)

log_prob <- predict(
  log_model,
  newdata = data.frame(x_test_small),
  type = "response"
)

log_pred <- ifelse(log_prob > 0.5, "spam", "ham")
log_pred <- factor(log_pred, levels = levels(y_test))

log_cm <- confusionMatrix(log_pred, y_test)

# Ridge regression
ridge_cv <- cv.glmnet(
```

```
  x_train,
  y_train,
  family = "binomial",
  alpha = 0,
  nfolds = 5
)

ridge_pred <- predict(
  ridge_cv,
  newx = x_test,
  s = "lambda.min",
  type = "class"
)

ridge_pred <- factor(ridge_pred[,1], levels = levels(y_test))
ridge_cm <- confusionMatrix(ridge_pred, y_test)

# Lasso regression
lasso_cv <- cv.glmnet(
  x_train,
  y_train,
  family = "binomial",
  alpha = 1,
  nfolds = 5
)

lasso_pred <- predict(
  lasso_cv,
  newx = x_test,
  s = "lambda.min",
  type = "class"
)

lasso_pred <- factor(lasso_pred[,1], levels = levels(y_test))
lasso_cm <- confusionMatrix(lasso_pred, y_test)

log_cm
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction ham spam
##       ham  949   65
##       spam  16   84
##
##                Accuracy : 0.9273
##                  95% CI : (0.9104, 0.9418)
##     No Information Rate : 0.8662
##     P-Value [Acc > NIR] : 7.181e-11
##
##                   Kappa : 0.6355
##
##  Mcnemar's Test P-Value : 9.643e-08
```

```
##
##             Sensitivity : 0.9834
##             Specificity : 0.5638
##          Pos Pred Value : 0.9359
##          Neg Pred Value : 0.8400
##              Prevalence : 0.8662
##          Detection Rate : 0.8519
##    Detection Prevalence : 0.9102
##       Balanced Accuracy : 0.7736
##
##        'Positive' Class : ham
##
```

```
ridge_cm
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction ham spam
##       ham  964   85
##       spam   1   64
##
##                Accuracy : 0.9228
##                  95% CI : (0.9055, 0.9378)
##     No Information Rate : 0.8662
##     P-Value [Acc > NIR] : 1.881e-09
##
##                   Kappa : 0.5626
##
##  Mcnemar's Test P-Value : < 2.2e-16
##
##             Sensitivity : 0.9990
##             Specificity : 0.4295
##          Pos Pred Value : 0.9190
##          Neg Pred Value : 0.9846
##              Prevalence : 0.8662
##          Detection Rate : 0.8654
##    Detection Prevalence : 0.9417
##       Balanced Accuracy : 0.7142
##
##        'Positive' Class : ham
##
```

```
lasso_cm
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction ham spam
##       ham  957   37
##       spam   8  112
##
##                Accuracy : 0.9596
```

```
##                  95% CI : (0.9463, 0.9704)
##     No Information Rate : 0.8662
##     P-Value [Acc > NIR] : < 2.2e-16
##
##                   Kappa : 0.81
##
##  Mcnemar's Test P-Value : 2.993e-05
##
##             Sensitivity : 0.9917
##             Specificity : 0.7517
##          Pos Pred Value : 0.9628
##          Neg Pred Value : 0.9333
##              Prevalence : 0.8662
##          Detection Rate : 0.8591
##    Detection Prevalence : 0.8923
##       Balanced Accuracy : 0.8717
##
##        'Positive' Class : ham
##
```

```
log_accuracy  <- as.numeric(log_cm$overall["Accuracy"])
log_precision <- as.numeric(log_cm$byClass["Pos Pred Value"])
log_recall    <- as.numeric(log_cm$byClass["Sensitivity"])
log_f1        <- 2 * log_precision * log_recall / (log_precision + log_recall)

ridge_accuracy  <- as.numeric(ridge_cm$overall["Accuracy"])
ridge_precision <- as.numeric(ridge_cm$byClass["Pos Pred Value"])
ridge_recall    <- as.numeric(ridge_cm$byClass["Sensitivity"])
ridge_f1        <- 2 * ridge_precision * ridge_recall / (ridge_precision + ridge_recall)

lasso_accuracy  <- as.numeric(lasso_cm$overall["Accuracy"])
lasso_precision <- as.numeric(lasso_cm$byClass["Pos Pred Value"])
lasso_recall    <- as.numeric(lasso_cm$byClass["Sensitivity"])
lasso_f1        <- 2 * lasso_precision * lasso_recall / (lasso_precision + lasso_recall)
```

The reduced logistic regression model achieved an accuracy of 0.927, while ridge regression achieved 0.923 and lasso regression achieved 0.96. Among the three methods, lasso regression produced the highest overall accuracy.

More importantly, lasso also performed best at identifying the minority *spam* class. Its recall was 0.992, compared with 0.983 for the reduced logistic regression model and 0.999 for ridge regression. Likewise, lasso obtained the highest precision, 0.963, indicating that a larger proportion of the messages it classified as *spam* were truly *spam*.

The performance of lasso is not surprising because lasso performs variable selection in addition to regularization. In a high-dimensional setting with 8235 predictors, many terms are likely irrelevant or weakly related to the response. By shrinking unimportant coefficients to zero, lasso is able to focus on the most informative words and avoid overfitting. Ridge regression, in contrast, keeps all predictors in the model, which may reduce its ability to distinguish *spam* from *ham* messages.

Next, we can investigate whether reducing the dimensionality of the feature matrix improves performance further.

```r
# Truncated SVD
svd_fit <- irlba(x_train, nv = 50)

x_train_svd <- x_train %*% svd_fit$v
x_test_svd  <- x_test %*% svd_fit$v

# Logistic regression on SVD features
svd_model <- glm(
  y_train ~ .,
  data = data.frame(y_train, x_train_svd),
  family = "binomial"
)

svd_prob <- predict(
  svd_model,
  newdata = data.frame(x_test_svd),
  type = "response"
)

svd_pred <- ifelse(svd_prob > 0.5, "spam", "ham")
svd_pred <- factor(svd_pred, levels = levels(y_test))

svd_cm <- confusionMatrix(svd_pred, y_test)

# Random projection using a random subset of 50 columns
set.seed(123)
proj_cols <- sample(1:ncol(x_train), 50)

x_train_proj <- x_train[, proj_cols]
x_test_proj  <- x_test[, proj_cols]

proj_model <- glm(
  y_train ~ .,
  data = data.frame(y_train, x_train_proj),
  family = "binomial"
)

proj_prob <- predict(
  proj_model,
  newdata = data.frame(x_test_proj),
  type = "response"
)

proj_pred <- ifelse(proj_prob > 0.5, "spam", "ham")
proj_pred <- factor(proj_pred, levels = levels(y_test))

proj_cm <- confusionMatrix(proj_pred, y_test)


svd_accuracy  <- svd_cm$overall["Accuracy"]
svd_precision <- svd_cm$byClass["Pos Pred Value"]
svd_recall    <- svd_cm$byClass["Sensitivity"]
svd_f1        <- 2 * svd_precision * svd_recall / (svd_precision + svd_recall)
```

```r
proj_accuracy  <- proj_cm$overall["Accuracy"]
proj_precision <- proj_cm$byClass["Pos Pred Value"]
proj_recall    <- proj_cm$byClass["Sensitivity"]
proj_f1        <- 2 * proj_precision * proj_recall / (proj_precision + proj_recall)

svd_accuracy
```

```
##  Accuracy
## 0.9156194
```

```r
svd_precision
```

```
## Pos Pred Value
##      0.9282203
```

```r
svd_recall
```

```
## Sensitivity
##   0.9782383
```

```r
svd_f1
```

```
## Pos Pred Value
##      0.9525732
```

```r
proj_accuracy
```

```
##  Accuracy
## 0.8761221
```

```r
proj_precision
```

```
## Pos Pred Value
##      0.8790101
```

```r
proj_recall
```

```
## Sensitivity
##   0.9937824
```

```r
proj_f1
```

```
## Pos Pred Value
##      0.9328794
```

The logistic regression model based on truncated SVD achieved an accuracy of 0.916, which is lower than the lasso model but still reasonably strong. Its precision and recall were 0.928 and 0.978, respectively. This suggests that reducing the original 8235 predictors to only 50 latent components preserved much of the predictive information in the dataset.

By contrast, the simple random projection approach performed significantly worse. Its accuracy dropped to 0.876, with precision 0.879 and recall 0.994. Although the projected model still classified most messages correctly, it lost more information than the SVD-based approach.

The difference between the two methods is within expectation. Truncated SVD constructs new features by capturing the major directions of variation in the TF–IDF matrix, whereas the random projection used here simply selected 50 columns at random. As a result, SVD retained more of the important structure in the data and produced stronger predictive performance.

These results indicate that moderate dimension reduction can substantially simplify the dataset without dramatically reducing classification accuracy. However, among all models considered so far, lasso regression still provides the strongest overall performance.

The next step is to determine whether a nonlinear kernel method can improve upon the linear approaches.

```
# Use SVD features
svd_train <- as.data.frame(x_train_svd)
svd_test  <- as.data.frame(x_test_svd)

svd_train$label <- y_train
svd_test$label  <- y_test

# Linear SVM
svm_linear <- svm(
  label ~ .,
  data = svd_train,
  kernel = "linear"
)

linear_pred <- predict(svm_linear, newdata = svd_test)

linear_cm <- confusionMatrix(linear_pred, y_test, positive = "spam")

# RBF Kernel SVM
svm_rbf <- svm(
  label ~ .,
  data = svd_train,
  kernel = "radial"
)

rbf_pred <- predict(svm_rbf, newdata = svd_test)

rbf_cm <- confusionMatrix(rbf_pred, y_test, positive = "spam")

# Extract metrics
linear_accuracy  <- as.numeric(linear_cm$overall["Accuracy"])
linear_precision <- as.numeric(linear_cm$byClass["Pos Pred Value"])
linear_recall    <- as.numeric(linear_cm$byClass["Sensitivity"])
linear_f1        <- 2 * linear_precision * linear_recall / (linear_precision + linear_recall)

rbf_accuracy  <- as.numeric(rbf_cm$overall["Accuracy"])
```

```r
rbf_precision <- as.numeric(rbf_cm$byClass["Pos Pred Value"])
rbf_recall    <- as.numeric(rbf_cm$byClass["Sensitivity"])
rbf_f1        <- 2 * rbf_precision * rbf_recall / (rbf_precision + rbf_recall)

linear_accuracy
```

```
## [1] 0.9129264
```

```r
linear_precision
```

```
## [1] 0.754902
```

```r
linear_recall
```

```
## [1] 0.5167785
```

```r
linear_f1
```

```
## [1] 0.6135458
```

```r
rbf_accuracy
```

```
## [1] 0.9479354
```

```r
rbf_precision
```

```
## [1] 0.8956522
```

```r
rbf_recall
```

```
## [1] 0.6912752
```

```r
rbf_f1
```

```
## [1] 0.780303
```

The linear support vector machine achieved an accuracy of 0.913, with precision 0.755 and recall 0.517. While its overall accuracy is reasonably high, its recall for the *spam* class is relatively low, indicating that the linear decision boundary fails to identify a substantial portion of *spam* messages.

By contrast, the kernel-based (RBF) support vector machine performed noticeably better. It achieved an accuracy of 0.948, with precision 0.896 and recall 0.691. In particular, the recall improved substantially compared to the linear model, suggesting that the nonlinear kernel is better able to capture patterns that distinguish spam from ham messages.

This difference demonstrates the importance of modeling nonlinear structure in high-dimensional text data. While linear methods are computationally efficient and perform well in many cases, they may fail to capture more complex relationships between words and message labels. The RBF kernel implicitly maps the data into a higher-dimensional feature space, allowing for more flexible decision boundaries and improved classification performance.

Discussion

The results of this project demonstrate that high-dimensional text data can be effectively analyzed using modern statistical learning methods that emphasize scalability and regularization. The TF–IDF representation produced a sparse feature matrix with 8235 predictors, illustrating the challenges associated with high-dimensional data.

Among the linear models, lasso regression performed best, achieving the highest accuracy and strongest performance in identifying *spam* messages. This result illustrates the importance of variable selection in high-dimensional settings, where many predictors are irrelevant or weakly informative. By shrinking unimportant coefficients to zero, lasso is able to focus on the most informative terms and avoid overfitting.

Dimension reduction methods showed that it is possible to substantially reduce the number of predictors without overly sacrificing predictive performance. The truncated SVD approach retained much of the important structure in the data, while the simple random projection approach resulted in a larger loss of information.

Finally, kernel-based methods demonstrated that allowing for nonlinear relationships can improve classification performance. The RBF support vector machine achieved higher accuracy and recall than the linear SVM, indicating that nonlinear patterns are present in the data. However, this improvement comes at the cost of increased computational complexity and reduced interpretability.

Conclusively, these results resonate with several key themes: the importance of sparse representations, the effectiveness of regularization in high-dimensional settings, the trade-offs involved in dimension reduction, and the potential benefits of nonlinear learning methods. While no single method is optimal in all respects, lasso regression provides a strong balance between accuracy and interpretability, while kernel methods offer additional predictive power given that computational cost is less of a concern.

A limitation of this study is that it focuses on a single dataset, and the results may not generalize to other types of high-dimensional data. In addition, the dataset is relatively small compared to true large-scale applications, so computational differences between methods may be less pronounced. Future work could extend this analysis to larger datasets, incorporate more advanced feature representations, or explore distributed computing approaches for handling realistically large-scale data.

Reference

Bingham, Ella, and Heikki Mannila. *Random Projection in Dimensionality Reduction: Applications to Image and Text Data.* Proceedings of the Seventh ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, 2001, pp. 245–250.

Cortes, Corinna, and Vladimir Vapnik. *Support-Vector Networks.* Machine Learning, vol. 20, no. 3, 1995, pp. 273–297.

Hastie, Trevor, Robert Tibshirani, and Jerome Friedman. *The Elements of Statistical Learning: Data Mining, Inference, and Prediction.* 2nd ed., Springer, 2009.

James, Gareth, et al. *An Introduction to Statistical Learning: with Applications in R.* 2nd ed., Springer, 2021.

Manning, Christopher D., Prabhakar Raghavan, and Hinrich Schütze. *Introduction to Information Retrieval.* Cambridge UP, 2008.